A 4.1.2 Machine Learning hardware

A 4.1.2 Describe the hardware requirements for various scenarios where machine learning isdeployed.

The hardware required for machine learning varies enormously depending on what is being done, how large the model is, and where the computation happens. This page covers the full spectrum from a student's laptop to a warehouse-scale data centre, and explains how to match hardware to a scenario by considering processing power, storage capacity, and scalability.

Last updated: 6 June 2026

Key Questions

- What are the two distinct phases of machine learning deployment, and how do their hardware requirements differ.
- What are the three factors - processing, storage, and scalability - and why does each matter when selecting hardware?
- How does a GPU differ from a CPU in its architecture, and why does that difference matter for training deep learning models?
- What distinguishes an ASIC from an FPGA, and in what scenario would each be preferred?
- When is edge deployment more appropriate than cloud deployment, and what constraints drive that choice?

IB Command Terms

Describe

Give a detailed account of the features or characteristics. For hardware scenarios, this means: name the hardware, explain its relevant characteristics (processing, storage, scalability), and connect it to a specific deployment scenario.

What to include

For each hardware type: what it is, how its processing architecture works, its storage and scalability profile, and a concrete scenario where it is the appropriate choice. Generic statements like 'GPUs are fast' are insufficient without explaining why and for what.

Introduction

After learning about the different types of ML models, it's time to ask a key question: **What kind of computer does it take to run them?** The answer depends heavily on the task. A simple model might run on your laptop, while a massive one like ChatGPT requires a supercomputer. In this section, we'll explore the spectrum of hardware that powers machine learning.

Training and inference: two different hardware problems

Every machine learning model goes through two distinct phases, each with fundamentally different hardware requirements. Understanding this distinction is essential for matching hardware to scenarios.

TRAINING	**INFERENCE**
The process of teaching the model by feeding it large amounts of data and adjusting its parameters to minimise error. Training is computationally intensive, typically involving billions of mathematical operations per second over hours, days, or weeks. • Processing: very high demand - massively parallel computation required. • Storage: very high demand - entire datasets must be accessible at speed; model checkpoints must be saved. • Scalability: critical - training time can be reduced by distributing work across multiple processors.	The process of using an already trained model to make predictions on new data. Inference is far less computationally demanding than training, but it may need to happen very quickly (low latency) or on hardware with severe power and size constraints. • Processing: moderate - a single forward pass through the model; no backpropagation. • Storage: low to moderate - only the trained model parameters need to be stored, not the full training dataset. • Scalability: variable - may need to scale to handle millions of simultaneous requests, or may run on a single low-power device.

The Three Factors

Processing: the raw computational power available - how many operations per second the hardware can perform, and whether it can perform them in parallel.

Storage: the capacity to hold training data, model parameters, and intermediate results - and critically, the speed at which that data can be read into the processor.

Scalability: the ability to add more hardware to reduce training time or handle more inference requests - whether through multiple GPUs in one machine, or thousands of machines working together.

The Hardware Spectrum

Hardware for machine learning ranges from general-purpose processors found in any computer to chips designed and manufactured for a single task. Each point on this spectrum involves trade-offs between processing power, flexibility, cost, and power consumption.

Central Processing Unit (CPU)

What it is: The general-purpose processor in any standard computer or laptop. A CPU has a small number of powerful cores, typically 4 to 32, each designed to execute complex, varied instructions sequentially at high speed.

Processing: Sequential execution excels at tasks that require complex logic and branching. Poorly suited to the highly parallel, repetitive matrix multiplications that dominate ML training.

Storage: Reads from system RAM and local disk. RAM is fast but limited in size; disk access is slow. For ML, large datasets may not fit in RAM.

Scalability: Limited. Adding more CPUs provides marginal gains for ML training due to communication overhead.

Best for training?: Only for small datasets and simple models. Acceptable for learning and experimentation.

Best for inference?: Yes, for models that are not computationally intensive and where latency is not critical - for example, running a trained text classifier on a server.

Scenario: A student running a logistic regression model on a CSV file of 5,000 rows. A small business running batch inference on a simple model overnight.

Graphics Processing Unit (GPU)

What it is: Originally designed to render graphics, a GPU contains thousands of simpler cores that operate in parallel. NVIDIA's A100 GPU, commonly used for ML, contains 6,912 CUDA cores. This architecture is exceptionally well-matched to the matrix multiplications that dominate neural network computation.

Processing: Extremely high throughput for parallel workloads. A GPU can perform the same operation on thousands of data values simultaneously, making it hundreds of times faster than a CPU for training neural networks.

Storage: GPU memory (VRAM) is separate from system RAM and typically ranges from 16 GB to 80 GB per card. VRAM capacity is a key constraint - model parameters and a batch of training data must fit simultaneously. High memory bandwidth allows data to be fed to the cores rapidly.

Scalability: Good. Multiple GPUs can be connected within a single machine, or across machines in a cluster. GPU clusters form the backbone of cloud ML infrastructure.

Best for training?: Yes - the primary hardware for training deep learning models.

Best for inference?: Yes, especially for high-throughput batch inference or real-time inference where many requests arrive simultaneously.

Scenario: A research team training a convolutional neural network on 1 million images, using four GPUs in a single workstation. A company running a recommendation system that must process thousands of user requests per second.

Tensor Processing Unit (TPU)

What it is: A custom chip designed by Google specifically to accelerate the tensor operations used in neural networks. A TPU is a type of ASIC - it was designed from scratch for one purpose. Unlike a GPU (which was adapted from graphics rendering), every transistor in a TPU exists to accelerate ML computation.

Processing: Faster and more power-efficient than a GPU for the specific matrix and tensor operations used in deep learning. Performance advantage is largest for the TensorFlow framework, for which TPUs were designed.

Storage: High-bandwidth memory integrated into the chip design, enabling very fast data movement between memory and processing units - a critical bottleneck in large model training.

Scalability: Excellent. Google operates TPU Pods - interconnected arrays of thousands of TPU chips - designed for distributed training of the largest models. Communication between

chips is handled by dedicated high-speed interconnects.

Best for training?: Yes, particularly for very large models trained on Google Cloud.

Best for inference?: Yes - efficient for high-volume inference in production.

Scenario: Google training a new large language model. A company using Google Cloud's ML APIs for large-scale natural language processing.

Application-Specific Integrated Circuit (ASIC) —

What it is: A chip designed and manufactured to perform one specific task permanently. The logic of the chip is fixed at the point of manufacture and cannot be changed. A TPU is an ASIC. Other examples include chips designed for Bitcoin mining and chips in smart doorbells that run face recognition.

Processing: Highest possible performance for the designated task. Because every circuit is optimised for that one operation, there is no wasted capacity on general-purpose logic. Correspondingly, an ASIC is completely useless for any task other than the one it was designed for.

Storage: Fixed at design time. The memory configuration is determined when the chip is designed and cannot be modified.

Scalability: Limited by design. ASICs are typically deployed as individual units performing their fixed task, not clustered for distributed computation.

Flexibility: None. If the task changes or the algorithm is updated, the ASIC cannot be reprogrammed. A new chip must be designed and manufactured.

Cost profile: Extremely high upfront development cost - designing and producing the first chip requires millions in engineering and fabrication. However, at manufacturing scale (millions of units), the per-unit cost is very low.

Scenario: A manufacturer producing 5 million smart doorbells, each needing to run face recognition locally without internet access. The fixed cost of designing the chip is recovered across the large production volume.

Field-Programmable Gate Array (FPGA) —

What it is: A chip containing an array of configurable logic blocks that can be connected and reconfigured by the user after manufacture. The hardware itself is reprogrammable - the same

physical chip can be reconfigured to implement different logic by loading a new configuration file.

Processing: High performance for the specific function it is currently configured for - lower than a dedicated ASIC for the same task, but far higher than a GPU for tasks that benefit from the FPGA's configurable interconnects and ultra-low latency.

Storage: Configurable on-chip memory; moderate capacity. External memory can be interfaced, but total storage is constrained compared to GPU setups.

Scalability: Moderate. FPGAs can be networked, but distributed FPGA systems are complex to design and programme.

Flexibility: Very high - the defining characteristic. The same chip can be reconfigured as the algorithm evolves, without manufacturing a new chip.

Cost profile: Moderate development cost. No fabrication required when the design changes - only a new configuration is loaded. More expensive per unit than an ASIC at scale.

Scenario: A financial trading firm prototyping a new low-latency ML inference pipeline that requires microsecond response times. The algorithm is still being refined, so the ability to reprogram the hardware without manufacturing a new chip is essential.

Edge devices

What they are: Small, low-power hardware units designed to run inference at the location where data is generated, without sending that data to a remote server. Edge devices may use a dedicated inference chip (a type of ASIC), a mobile GPU, or a small FPGA. The defining characteristic is local execution under tight power and size constraints.

Processing: Low to moderate - optimised for efficiency rather than peak throughput. Sufficient for running a pre-trained model; not sufficient for training.

Storage: Severely constrained. Only the trained model parameters and a small buffer for incoming data need to be stored. Large datasets cannot be held locally.

Scalability: None in isolation - a single edge device handles only its own data. However, systems of many edge devices (a network of smart cameras, for example) can collectively process large volumes of data in a distributed manner.

Key advantages: No network dependency - inference works without internet access. Low latency - data does not travel to a remote server and back. Privacy - sensitive data (medical images, face scans) never leaves the device.

Scenario: A smart doorbell running face recognition to identify registered visitors locally. A medical tablet in a remote clinic running a pre-trained diagnostic model on patient scans without internet connectivity. An autonomous vehicle processing sensor data in real time.

Cloud platforms and HPC centres

What they are: Cloud platforms (such as Amazon Web Services, Google Cloud, and Microsoft Azure) provide on-demand access to large clusters of GPUs, TPUs, and other hardware via the internet. HPC (High-Performance Computing) centres are large, centralised facilities - often at universities or national research institutions - housing supercomputers for the same purpose. Both provide effectively unlimited processing capacity compared to owned hardware.

Processing: Effectively unlimited - a user can rent hundreds or thousands of GPUs simultaneously. The processing scale is determined by budget, not physical hardware ownership.

Storage: Petabyte-scale distributed storage systems. Datasets, model checkpoints, and results can be stored at any scale. Storage is separate from compute and can be accessed by any machine in the cluster.

Scalability: Excellent - the defining advantage. Resources can be scaled up instantly for a large training run and scaled back down when not needed, avoiding the cost of owning idle hardware.

Cost profile: Pay-as-you-go on cloud platforms; no capital expenditure. Operating costs can be significant for large training runs. HPC centres typically require a research justification and queue time rather than direct payment.

Key advantage over owned hardware: Eliminates the need for a large upfront investment in hardware that may be idle most of the time. A startup can train a large model for the cost of a few days of GPU rental rather than purchasing a server farm.

Scenario: A startup training a large language model for the first time, using 128 rented GPUs for 48 hours. A university research group training a climate simulation model that requires more compute than their local servers can provide.

Hardware comparison: processing, storage, and scalability

The table below summarises all seven hardware configurations against the three factors specified in the learning statement, plus flexibility, cost, and best use. Use it for revision and scenario selection.

HARDWARE	PROCESSING	STORAGE	SCALABILITY	FLEXIBILITY	COST PROFILE
CPU (standard laptop)	Low - few powerful cores, sequential	Limited by system RAM and local disk	Poor - single machine only	Very high - general purpose	Low capital cost; high opportunity cost for large tasks
GPU	Very high - thousands of parallel cores	GPU memory (VRAM) is a key constraint; fast bandwidth	Good - multiple GPUs can be clustered	High - programmable for many ML tasks	High hardware cost; widely available on cloud
TPU	Extremely high for tensor operations; purpose-built	High-bandwidth memory integrated into chip design	Excellent - designed for large-scale distributed training	Low - optimised for specific frameworks (TensorFlow)	High; available via Google Cloud
ASIC	Highest possible for its designated task	Fixed; determined at design time	Limited; designed for a specific throughput	None - fixed function, cannot be reprogrammed	Extremely high development cost; very low per-unit cost at scale
FPGA	High; lower than ASIC for same task	On-chip memory configurable; moderate capacity	Moderate; can be networked but complex	Very high - logic gates reprogrammable after manufacture	Moderate development cost; no fabrication required
Edge device	Low to moderate; optimised for efficiency not peak throughput	Minimal - constraied by physical size and power budget	None - single device; no distributed capability	Low - usually fixed hardware running a pre-trained model	Low per-unit cost; no ongoing infrastructure cost

HARDWARE	PROCESSING	STORAGE	SCALABILITY	FLEXIBILITY	COST PROFILE
Cloud platform / HPC centre	Effectively unlimited - rent as many GPUs/TPUs as needed	Petabyte-scale object storage; distributed file systems	Excellent - scale up or down on demand	Very high - access any hardware type as a service	Pay-as-you-go; no capital cost; high operating cost at scale

Selecting hardware for a scenario

Exam questions will present a scenario and ask you to identify and justify the most appropriate hardware. The key is to read the scenario for clues: Is this training or inference? What are the constraints - power, size, connectivity, budget, latency? The table below models the reasoning process.

SCENARIO	MOST SUITABLE HARDWARE	JUSTIFICATION
A student learning ML, running a simple decision tree on 1,000 rows of data	**Standard laptop (CPU)**	*Dataset is small; model is simple; no parallel computation needed. A CPU handles this comfortably.*
A research team training a large convolutional neural network on 10 million labelled images	**Cloud platform with GPU cluster (or HPC centre)**	*Training requires massive parallel computation. A single GPU is insufficient; a cluster of hundreds of GPUs accessed via cloud or HPC is standard practice.*
Google training a new version of its Gemini language model	**TPU cluster (cloud-scale)**	*The scale requires purpose-built tensor hardware with extremely high memory bandwidth. Google's TPU pods are designed specifically for this workload.*
A manufacturer deploying face recognition on one million smart doorbells	**Edge device (custom ASIC)**	*Inference must happen locally without network dependency. A custom ASIC provides maximum efficiency and low power at manufacturing scale; no internet connection required.*

SCENARIO	MOST SUITABLE HARDWARE	JUSTIFICATION
A startup prototyping a new ML architecture for low-latency financial trading	**FPGA**	*The architecture is not yet finalised so flexibility is essential. FPGAs can be reprogrammed as the design evolves, and they provide the microsecond-level latency trading requires.*
A hospital deploying a trained skin cancer classifier on a dermatology tablet	**Edge device (mobile ASIC or GPU)**	*Inference must run locally to protect patient data and function in areas with unreliable connectivity. Low power consumption and fast response time are the key requirements.*
A small company that needs to train a sentiment analysis model but cannot afford GPU servers	**Cloud platform (rented GPUs)**	*Cloud platforms eliminate upfront capital cost. The company pays only for the hours of GPU time used during training, then runs inference on cheaper hardware.*

How to approach a scenario question

Step 1: Identify whether the scenario describes training or inference. Training requires much more processing power and storage.

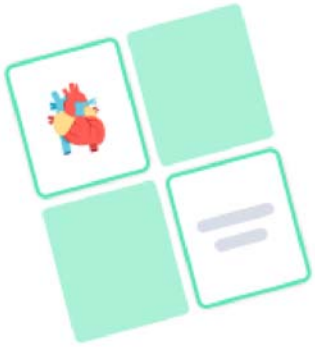
Step 2: Identify the constraints. Look for keywords: 'without internet' points to edge; 'millions of units' points to ASIC; 'limited budget' points to cloud; 'prototype' or 'changing design' points to FPGA; 'massive scale' and 'large model' point to GPU cluster or TPU.

Step 3: State the hardware, explain why it matches the processing requirement, comment on the storage and scalability implications, and connect it explicitly to the scenario.

A 4.1.2 Quizlet

Use this Quizlet to learn the key terms from this section. You can try it in different modes to make it more fun.

Match A 4.1.2 Machine Learning Hardware



Ready to play?

Match all the terms with their definitions as fast as you can. Avoid wrong matches, they add extra time!

Start game

View this flashcard set Choose a Study Mode

A 4.1.2 Quiz

Try this quiz out to test your knowledge of this section.

1

What is the primary architectural advantage of a GPU over a CPU for training deep learning models?

A. It uses less electricity.

B. It has thousands of cores for parallel processing.

C. It has a higher clock speed.

D. It is better at sequential tasks.

2

The process of using a fully trained model to make a prediction on new data is called:

A. Clustering

B. Inference

C. Training

D. Compiling

3

A company is manufacturing a million smart doorbells that need to perform face recognition instantly on the device itself. Which hardware would be most suitable for this task?

A. An Edge Device (like a specialised ASIC)

B. A standard CPU

C. A High-Performance Computing (HPC) Center

D. A Cloud Platform with GPUs

4

A university research team needs to train a massive new language model from scratch. What is their most likely hardware configuration?

A. An FPGA.

B. A cluster of hundreds of GPUs on a Cloud Platform.

C. An Edge Device.

D. A single high-end laptop.

5

A chip that is designed for one single, permanent task to achieve the highest possible performance is known as:

A. A GPU

B. An ASIC

C. An FPGA

D. A CPU

6

What is the key characteristic of an FPGA that distinguishes it from an ASIC?

A. It is only used for training models.

B. It can be reconfigured after manufacturing.

C. It is faster than an ASIC.

D. It consumes less power than an ASIC.

7

Google's TPU is a specialised type of which hardware category?

A. GPU

B. ASIC

C. CPU

D. FPGA

8

Which of these factors is LEAST important for an Edge Device performing inference?

A. Small physical size

B. Low power consumption

C. The ability to train new models from scratch

D. Fast response time (low latency)

9

Why would a startup with a limited budget choose to rent computing power from a Cloud Platform instead of building its own HPC Centre?

A. Cloud platforms are always faster.

B. HPC Centre cannot be used for machine learning.

C. It avoids a massive upfront cost and provides scalability.

D. loud platforms are the only place to get GPUs.

10

The process of "Training" a model is typically associated with which hardware environment?

A. Edge Devices

B. Standard Laptops

C. FPGAs

D. Cloud Platforms / HPC Centers

All materials on this website are for the exclusive use of teachers and students at subscribing schools for the period of their subscription. Any unauthorised copying or posting of materials on other websites is an infringement of our copyright and could result in your account being blocked and legal action being taken against you.

Report a problem